Résolution du Problème du Voyageur de Commerce : Étude Comparative entre Algorithme Génétique et Approche Hybride

Module : Métaheuristique

Année universitaire : 2025 – 2026

Master IARO

Réalisé par : Hafsa Zarhouti

Ce rapport analyse la résolution du problème du voyageur de commerce (TSP) à l'aide de différentes approches heuristiques et métaheuristiques. Il présente l'implémentation et la comparaison de l'algorithme Nearest Neighbor, d'un algorithme génétique et d'une approche hybride.

9 mars 2026

1 Introduction

Le problème du voyageur de commerce (Traveling Salesman Problem – TSP) est l'un des problèmes d'optimisation combinatoire les plus étudiés en recherche opérationnelle et en informatique. Il consiste à déterminer le plus court circuit permettant à un voyageur de visiter un ensemble de villes exactement une seule fois avant de revenir à la ville de départ. Chaque paire de villes est associée à une distance ou un coût de déplacement, et l'objectif est de minimiser la longueur totale du trajet.

Le TSP est connu pour être un problème NP-difficile. Cela signifie qu'il n'existe pas, à ce jour, d'algorithme exact capable de résoudre efficacement toutes les instances du problème lorsque leur taille devient importante. En effet, le nombre de solutions possibles augmente de manière factorielle avec le nombre de villes. Par exemple, pour un problème comportant n villes, le nombre de circuits possibles est de l'ordre de $(n-1)!/2$. Cette explosion combinatoire rend les méthodes exactes rapidement impraticables pour les grandes instances.

Pour contourner cette difficulté, de nombreuses méthodes heuristiques et métaheuristiques ont été proposées dans la littérature. Les heuristiques permettent d'obtenir rapidement des solutions de bonne qualité, sans garantir nécessairement l'optimalité. Parmi les heuristiques les plus simples pour le TSP figure l'algorithme du *Nearest Neighbor* (NN), qui construit progressivement une tournée en choisissant à chaque étape la ville non visitée la plus proche.

Les métaheuristiques, quant à elles, constituent une famille d'algorithmes d'optimisation inspirés de phénomènes naturels ou de principes généraux d'exploration de l'espace des solutions. Elles sont particulièrement efficaces pour traiter des problèmes complexes tels que le TSP. Parmi ces approches, les algorithmes génétiques (Genetic Algorithms – GA) occupent une place importante. Inspirés des mécanismes de l'évolution naturelle, ils utilisent des opérateurs tels que la sélection, le croisement et la mutation afin d'explorer l'espace des solutions et d'améliorer progressivement la qualité des tournées générées.

Cependant, les métaheuristiques globales comme les algorithmes génétiques peuvent parfois produire des solutions qui restent éloignées de l'optimum local. Pour améliorer la qualité des solutions obtenues, il est courant de combiner ces approches avec des méthodes de recherche locale. Dans ce contexte, l'opérateur d'amélioration locale *2-opt* est largement utilisé pour le TSP. Cette méthode consiste à remplacer deux arêtes d'une tournée par deux nouvelles arêtes afin de réduire la longueur totale du circuit.

Dans ce projet, nous proposons d'étudier et de comparer plusieurs approches pour la résolution du problème du voyageur de commerce. Plus précisément, trois méthodes sont implémentées et analysées :

- une heuristique constructive basée sur l'algorithme du plus proche voisin (Nearest Neighbor) ;
- un algorithme génétique permettant d'explorer l'espace des solutions par évolution d'une population de tournées ;
- une approche hybride combinant l'algorithme génétique avec une procédure d'amélioration locale de type 2-opt.

Les performances de ces différentes méthodes sont évaluées sur plusieurs instances issues de la bibliothèque TSPLIB, qui constitue une référence dans l'étude expérimentale du problème du voyageur de commerce. Les expériences réalisées permettent de comparer la qualité des solutions obtenues, la vitesse de convergence des algorithmes ainsi que leur temps d'exécution.

L'objectif principal de ce travail est donc d'analyser l'impact de l'hybridation entre une métaheuristique globale et une méthode de recherche locale dans la résolution du TSP. Nous cherchons en particulier à déterminer dans quelle mesure l'ajout d'une phase d'amélioration locale peut améliorer la qualité des solutions produites par l'algorithme génétique.

La suite de ce rapport est organisée comme suit. Dans le chapitre suivant, nous présentons le problème du voyageur de commerce ainsi que les instances utilisées dans les expérimentations. Ensuite, nous décrivons les différentes méthodes implémentées, notamment l'heuristique Nearest Neighbor, l'algorithme génétique et la procédure d'amélioration locale 2-opt. Nous présentons ensuite le protocole expérimental ainsi que les résultats obtenus. Enfin, nous analysons ces résultats avant de conclure et de proposer quelques perspectives d'amélioration.

2 Présentation du problème du voyageur de commerce

2.1 Définition du problème

Le problème du voyageur de commerce (*Traveling Salesman Problem – TSP*) est un problème classique d'optimisation combinatoire. Il consiste à déterminer le plus court circuit permettant de visiter un ensemble de villes exactement une seule fois avant de revenir à la ville de départ.

Soit un ensemble de n villes noté $V = \{1, 2, \ldots, n\}$. La distance entre chaque paire de villes i et j est représentée par une matrice des distances $D = (d_{ij})$, où d_{ij} correspond au coût ou à la distance entre les villes i et j.

L'objectif du problème est de trouver une tournée passant par toutes les villes une seule fois et minimisant la longueur totale du trajet. Pour modéliser ce problème, on introduit une variable de décision binaire x_{ij} définie comme suit :

$$x_{ij} = \begin{cases} 1 & \text{si l'arête reliant la ville } i \text{ à la ville } j \text{ est utilisée dans la tournée,} \\ 0 & \text{sinon.} \end{cases}$$

La formulation mathématique du TSP peut alors être exprimée comme suit :

$$\min \sum_{i=1}^{n} \sum_{j=1}^{n} d_{ij} x_{ij}$$

sous les contraintes suivantes :

— chaque ville doit être visitée exactement une fois ;

— chaque ville doit avoir exactement une arête entrante et une arête sortante ;

— la solution doit former une seule tournée couvrant toutes les villes.

Cette formulation permet de modéliser le problème de manière rigoureuse. Cependant, le TSP appartient à la classe des problèmes NP-difficiles, ce qui signifie que la recherche d'une solution optimale devient très coûteuse en temps de calcul lorsque le nombre de villes augmente. Pour cette raison, les heuristiques et les métaheuristiques sont souvent utilisées afin d'obtenir des solutions de bonne qualité dans un temps raisonnable.

2.2 Instances utilisées (TSPLIB)

Afin d'évaluer les performances des métaheuristiques développées dans ce projet, plusieurs instances de référence du problème du voyageur de commerce ont été utilisées. Ces instances proviennent de la bibliothèque **TSPLIB**, qui constitue une base de données standard largement utilisée dans la recherche en optimisation combinatoire pour tester et comparer les algorithmes.

Chaque instance représente un ensemble de villes définies par leurs coordonnées géographiques ou par une matrice de distances. L'objectif est de déterminer le circuit de longueur minimale visitant chaque ville une seule fois avant de revenir à la ville de départ.

Les instances sélectionnées couvrent différentes tailles de problèmes afin d'analyser le comportement des algorithmes sur des cas de complexité croissante.

Instance	**Nombre de villes**
burma14	14
ulysses16	16
ulysses22	22
att48	48
eil51	51
berlin52	52
st70	70
eil76	76
kroA100	100
ch130	130

Table 1 – Instances TSPLIB utilisées pour l'évaluation expérimentale

Ces instances ont été choisies afin de tester les algorithmes sur des problèmes de tailles variées, allant de petites instances (14 villes) jusqu'à des instances plus complexes dépassant 100 villes. Cette diversité permet d'évaluer la capacité des métaheuristiques à produire des solutions de bonne qualité tout en maintenant un temps de calcul raisonnable.

3 Méthodes Implémentées

Cette section présente les différentes méthodes développées pour résoudre le problème du voyageur de commerce. L'implémentation comprend la lecture des instances TSPLIB, la construction de la matrice des distances, une heuristique constructive, un algorithme génétique ainsi qu'une méthode d'amélioration locale.

3.1 Lecture des instances TSPLIB

Les instances du problème sont stockées dans des fichiers au format `.tsp`. Chaque fichier contient les coordonnées des villes qui composent l'instance.

Le processus de lecture comporte les étapes suivantes :

— ouverture du fichier `.tsp`

— lecture des coordonnées des villes

— stockage des coordonnées dans une structure de données

La fonction présentée dans la Figure 1 permet d'extraire les informations nécessaires à partir des fichiers TSPLIB.

```python
def read_tsplib(filename):
    path = os.path.join("data", filename)
    coords = []

    with open(path, 'r') as f:
        lines = f.readlines()

    start = False
    for line in lines:
        if "NODE_COORD_SECTION" in line:
            start = True
            continue
        if "EOF" in line:
            break
        if start:
            parts = line.strip().split()
            if len(parts) >= 3:
                coords.append((float(parts[1]), float(parts[2])))

    return np.array(coords)
```

FIGURE 1 – Extrait du code Python pour la lecture des fichiers TSPLIB.

Rôle de la fonction :
— lire le fichier contenant l'instance
— extraire les coordonnées de chaque ville
— retourner la liste des villes utilisées par l'algorithme

3.2 Calcul de la matrice de distances

Une fois les coordonnées des villes extraites, il est nécessaire de calculer la distance entre chaque paire de villes.

La distance utilisée dans ce projet est la distance euclidienne définie par la formule suivante :

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

où :
— x_i, y_i représentent les coordonnées de la ville i
— x_j, y_j représentent les coordonnées de la ville j

Les distances sont ensuite stockées dans une matrice carrée de dimension $n \times n$, où n représente le nombre de villes.

La Figure 2 illustre la structure de cette matrice.

3.3 Heuristique Nearest Neighbor (NN)

L'heuristique du plus proche voisin est une méthode simple pour construire rapidement une solution initiale du problème TSP.

Le principe est le suivant :

1. choisir une ville de départ

```python
def euclidean_distance(a, b):
    return int(round(math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)))


def compute_distance_matrix(coords):
    n = len(coords)
    dist = np.zeros((n, n), dtype=int)
    for i in range(n):
        for j in range(n):
            if i != j:
                dist[i][j] = euclidean_distance(coords[i], coords[j])
    return dist
```

FIGURE 2 – Structure de la matrice de distances.

2. sélectionner la ville non visitée la plus proche
3. répéter le processus jusqu'à visiter toutes les villes

Pseudo-code :

```
Choisir une ville initiale
Tant qu'il reste des villes non visitées
    choisir la ville la plus proche
    ajouter au tour
Retourner au point de départ
```

La Figure 3 illustre l'application de l'heuristique du plus proche voisin sur une instance du problème TSP.

La complexité de cet algorithme est approximativement :

$$O(n^2)$$

où n représente le nombre de villes.

```python
def nearest_neighbor(dist):
    n = len(dist)
    unvisited = list(range(n))
    current = random.choice(unvisited)
    tour = [current]
    unvisited.remove(current)

    while unvisited:
        next_city = min(unvisited, key=lambda x: dist[current][x])
        tour.append(next_city)
        unvisited.remove(next_city)
        current = next_city

    return tour
```

FIGURE 3 – Illustration de l'heuristique Nearest Neighbor.

3.4 Algorithme Génétique (GA)

Les algorithmes génétiques sont des méthodes d'optimisation inspirées du processus de sélection naturelle.

Une population de solutions évolue au fil des générations afin d'améliorer progressivement la qualité des solutions.

3.4.1 Représentation

Une solution est représentée par une permutation des villes correspondant à l'ordre de visite dans le tour.

3.4.2 Population initiale

La population initiale est générée aléatoirement afin d'introduire une diversité dans l'espace des solutions. La Figure 4 présente la fonction utilisée pour générer la population initiale.

```python
def init_population(pop_size, n, dist=None):
    pop = []


    if dist is not None:
        pop.append(nearest_neighbor(dist))

    while len(pop) < pop_size:
        p = list(range(n))
        random.shuffle(p)
        pop.append(p)

    return pop
```

FIGURE 4 – Code Python pour la génération de la population initiale.

3.4.3 Sélection

La sélection est réalisée à l'aide de la méthode du tournoi qui permet de choisir les individus les plus adaptés. La Figure 5 illustre l'implémentation de cette étape.

3.4.4 Croisement

Le croisement permet de combiner deux solutions parentales pour produire de nouvelles solutions. La Figure 6 montre la fonction de croisement utilisée.

```python
def selection(pop, dist, k=3):
    selected = random.sample(pop, k)
    selected.sort(key=lambda x: tour_length(x, dist))
    return selected[0]
```

FIGURE 5 – Code Python pour la sélection par tournoi.

```python
def crossover(p1, p2):
    n = len(p1)
    a, b = sorted(random.sample(range(n), 2))
    child = [-1]*n
    child[a:b] = p1[a:b]

    ptr = 0
    for city in p2:
        if city not in child:
            while child[ptr] != -1:
                ptr += 1
            child[ptr] = city
    return child
```

FIGURE 6 – Code Python pour l'opération de croisement.

3.4.5 Mutation

La mutation introduit de la diversité génétique dans la population afin d'éviter la convergence prématurée. La Figure 7 présente l'opération de mutation.

```python
def mutation(tour, rate=0.1):
    if random.random() < rate:
        i, j = random.sample(range(len(tour)), 2)
        tour[i], tour[j] = tour[j], tour[i]
    return tour
```

FIGURE 7 – Code Python pour l'opération de mutation.

3.4.6 Évaluation

La qualité d'une solution est mesurée par la longueur totale du tour parcouru. Cette distance correspond à la somme des distances entre chaque paire de villes consécutives dans la permutation représentant la solution.

La Figure 8 présente la fonction utilisée pour calculer cette distance.

```python
def tour_length(tour, dist):
    n = len(tour)
    return sum(dist[tour[i]][tour[(i+1)%n]] for i in range(n))
```

FIGURE 8 – Fonction de calcul de la longueur d'un tour.

3.4.7 Implémentation de l'algorithme génétique

L'algorithme génétique constitue le mécanisme principal d'optimisation. Il combine les différentes opérations précédentes (sélection, croisement et mutation) afin de faire évoluer progressivement la population de solutions.

Le principe général est le suivant :

— génération d'une population initiale
— sélection des parents
— production de nouveaux individus par croisement
— introduction de diversité par mutation
— conservation des meilleures solutions (élitisme)

La Figure 9 présente l'implémentation de l'algorithme génétique utilisée dans ce travail.

```python
def genetic_algorithm(dist, pop_size=40, generations=80, hybrid=False):
    n = len(dist)
    pop = init_population(pop_size, n, dist)
    best_curve = []

    for _ in range(generations):
        new_pop = []

        # === ELITISME ===
        elite = min(pop, key=lambda x: tour_length(x, dist))
        new_pop.append(elite.copy())

        # === GENERATION DES ENFANTS ===
        while len(new_pop) < pop_size:
            p1 = selection(pop, dist)
            p2 = selection(pop, dist)
            child = crossover(p1, p2)
            child = mutation(child)
            new_pop.append(child)

        pop = new_pop
```

FIGURE 9 – Implémentation de l'algorithme génétique pour le TSP.

3.5 Amélioration locale : 2-opt

La méthode 2-opt est une technique d'amélioration locale couramment utilisée pour le problème du voyageur de commerce. Son objectif est d'améliorer une solution existante en réduisant la longueur totale du tour.

Le principe consiste à sélectionner deux arêtes du tour et à inverser le segment situé entre ces deux arêtes afin de réduire la distance totale. Si l'échange permet d'obtenir une solution plus courte, la modification est acceptée. Ce processus est répété jusqu'à ce qu'aucune amélioration ne soit possible.

La Figure 10 présente l'implémentation de l'algorithme 2-opt utilisée dans ce travail.

```python
def two_opt_fast(tour, dist):
    n = len(tour)
    improved = True

    while improved:
        improved = False
        for i in range(1, n-2):
            for j in range(i+1, n):
                if j-i == 1:
                    continue

                a, b = tour[i-1], tour[i]
                c, d = tour[j-1], tour[j % n]

                if dist[a][b] + dist[c][d] > dist[a][c] + dist[b][d]:
                    tour[i:j] = reversed(tour[i:j])
                    improved = True
        if improved:
            break

    return tour
```

FIGURE 10 – Implémentation de l'amélioration locale 2-opt.

3.6 Algorithme Hybride

Afin d'améliorer les performances de l'algorithme génétique, une approche hybride combinant l'algorithme génétique (GA) et la méthode d'amélioration locale 2-opt a été utilisée.

Le principe consiste à appliquer l'algorithme 2-opt uniquement sur la meilleure solution obtenue à chaque génération. Cette stratégie permet d'améliorer localement la qualité du meilleur tour sans augmenter significativement le coût de calcul.

Comme illustré dans la Figure 11, la meilleure solution de la population est d'abord identifiée à l'aide de la fonction `tour_length`. Si l'option d'hybridation est activée, l'algorithme 2-opt est ensuite appliqué à cette solution afin de réduire davantage la distance totale du tour.

Cette hybridation permet d'améliorer la qualité des solutions tout en conservant la capacité d'exploration globale de l'algorithme génétique.

4 Protocole Expérimental

Cette section décrit les paramètres utilisés pour les expériences et les conditions d'exécution des différents algorithmes.

```python
        # === Hybridation seulement sur le meilleur ===
        best = min(pop, key=lambda x: tour_length(x, dist))

        if hybrid:
            best = two_opt_fast(best.copy(), dist)

        best_curve.append(tour_length(best, dist))

    best = min(pop, key=lambda x: tour_length(x, dist))
    if hybrid:
        best = two_opt_fast(best.copy(), dist)

    return best, best_curve
```

FIGURE 11 – Implémentation de l'hybridation entre l'algorithme génétique et l'amélioration locale 2-opt.

Les expérimentations ont été réalisées sur plusieurs instances classiques de la bibliothèque TSPLIB. Les algorithmes comparés sont l'heuristique Nearest Neighbor (NN), l'algorithme génétique (GA) et l'approche hybride combinant GA et l'amélioration locale 2-opt.

Paramètre	Valeur
Taille de la population	40
Nombre de générations	80
Taux de mutation	0.1

TABLE 2 – Paramètres utilisés pour l'algorithme génétique

Afin de limiter l'influence de l'aléatoire inhérent aux algorithmes évolutionnaires, chaque instance a été exécutée **5 fois** et la moyenne des résultats obtenus a été calculée.

5 Résultats Expérimentaux

5.1 Comparaison des algorithmes

Le Tableau 3 présente les distances obtenues par les différentes méthodes ainsi que le gain obtenu par l'approche hybride par rapport à l'heuristique NN.

5.2 Temps d'exécution

Le Tableau 4 présente les temps d'exécution moyens observés pour les différents algorithmes.

5.3 Courbes de convergence

Les Figures 12 à 21 présentent les courbes de convergence obtenues pour chacune des instances étudiées. Chaque figure compare l'évolution du coût de la meilleure solution au

Instance	NN	GA	Hybrid	Gain (%)
att48	41649	37389	34842	16.34
eil51	556	479	488	12.23
berlin52	10200	8764	8986	11.90
st70	815	797	789	3.19
eil76	642	664	590	8.10
kroA100	26728	26610	22826	14.60
ch130	7602	7503	7264	4.45
burma14	40	31	30	25.00
ulysses16	91	73	74	18.68
ulysses22	95	75	76	20.00

TABLE 3 – Comparaison des performances des algorithmes

Instance	Temps NN (s)	Temps GA (s)	Temps Hybrid (s)
att48	0.0014	6.3527	6.5562
eil51	0.0000	6.3559	7.8733
berlin52	0.0021	7.7250	7.0582
st70	0.0000	10.1747	9.5065
eil76	0.0000	8.7519	10.9802
kroA100	0.0000	12.4861	15.0450
ch130	0.0081	7.1306	10.1533
burma14	0.0000	1.8722	1.9173
ulysses16	0.0000	2.1526	2.1115
ulysses22	0.0000	2.8098	2.8477

TABLE 4 – Temps d'exécution des algorithmes

cours des générations pour l'algorithme génétique (GA) et pour l'algorithme hybride (GA + 2-opt).

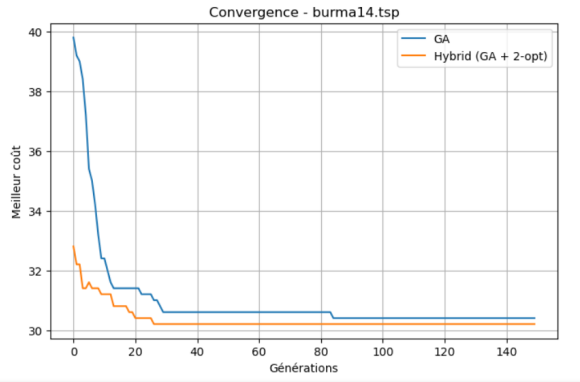

FIGURE 12 – Comparaison des courbes de convergence pour l'instance burma14

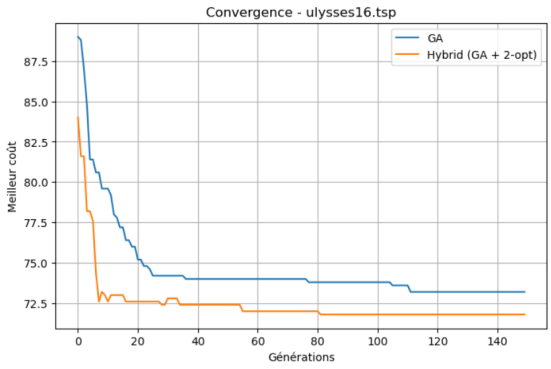

FIGURE 13 – Comparaison des courbes de convergence pour l'instance ulysses16

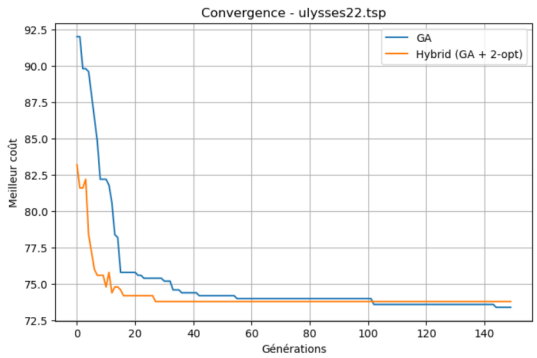

FIGURE 14 – Comparaison des courbes de convergence pour l'instance ulysses22

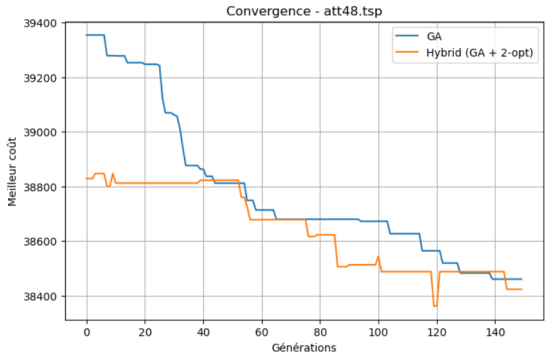

FIGURE 15 – Comparaison des courbes de convergence pour l'instance att48

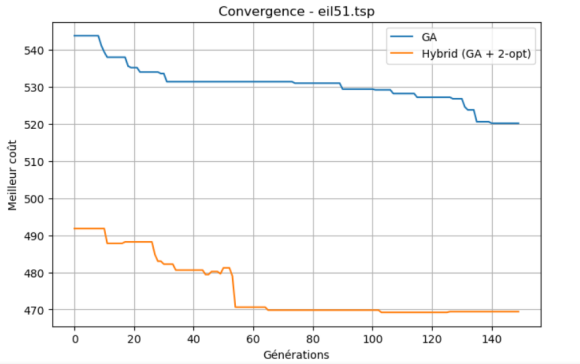

FIGURE 16 – Comparaison des courbes de convergence pour l'instance eil51

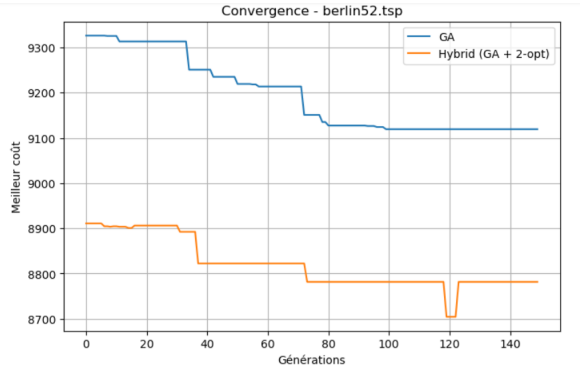

FIGURE 17 – Comparaison des courbes de convergence pour l'instance berlin52

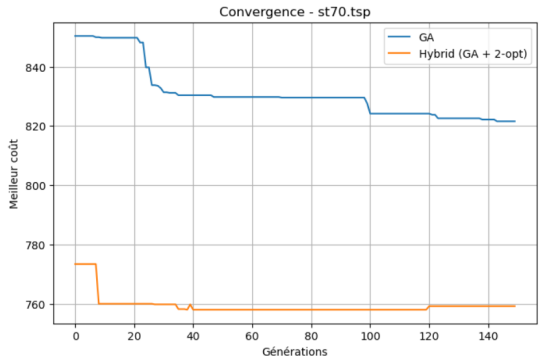

FIGURE 18 – Comparaison des courbes de convergence pour l'instance st70

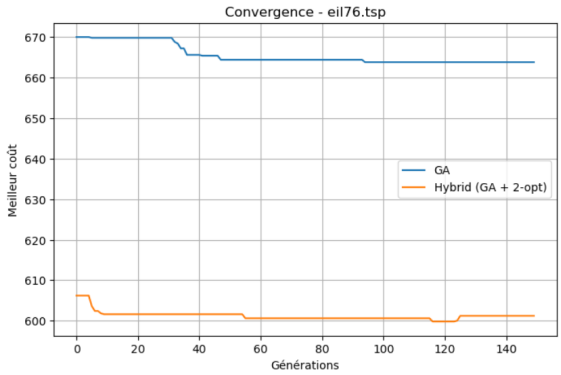

FIGURE 19 – Comparaison des courbes de convergence pour l'instance eil76

La Figure 22 présente une vue globale de la convergence en comparant les performances de l'algorithme génétique et de l'algorithme hybride sur l'ensemble des instances étudiées.

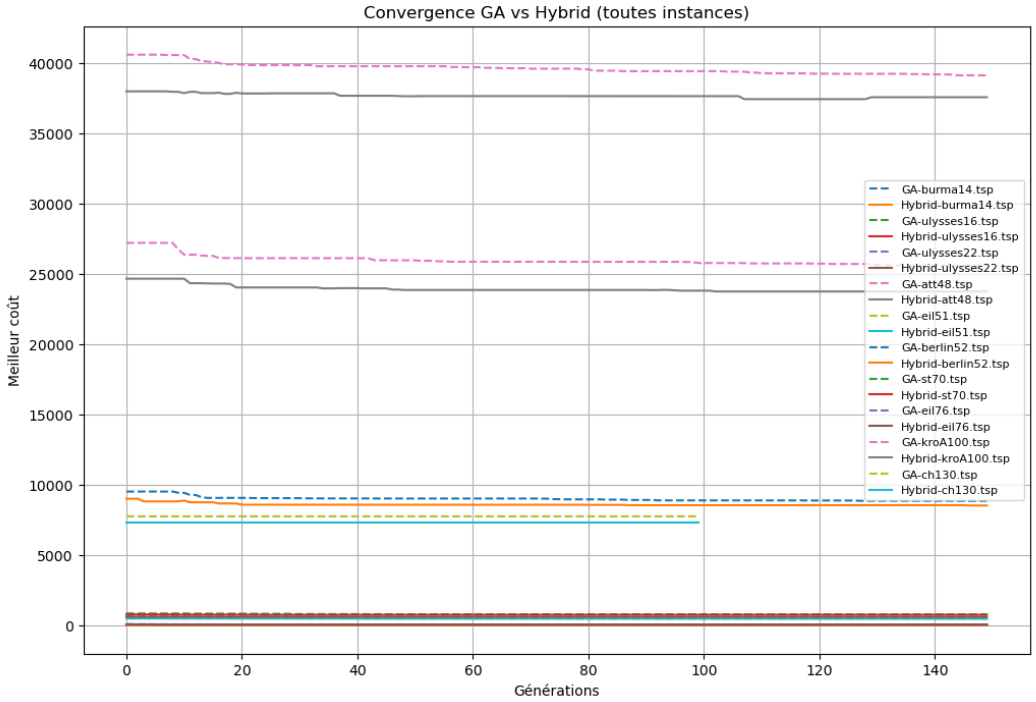

FIGURE 22 – Comparaison globale de la convergence entre GA et GA + 2-opt pour l'ensemble des instances

6 Analyse des Résultats

L'analyse des résultats permet d'observer le comportement des différents algorithmes sur les instances testées.

La Figure 22 présente l'évolution du meilleur coût obtenu au cours des générations pour l'ensemble des instances étudiées. Cette figure compare les performances de l'algorithme génétique (GA) et de l'approche hybride combinant GA et l'amélioration locale 2-opt.

De manière générale, on observe que l'approche hybride permet d'obtenir des solutions de meilleure qualité que l'algorithme génétique seul. En effet, pour la majorité des instances, les courbes correspondant à l'algorithme hybride se situent en dessous de celles de l'algorithme génétique, ce qui indique un coût de tournée plus faible.

Cette amélioration s'explique par l'utilisation de la recherche locale 2-opt, qui permet d'améliorer les solutions générées par l'algorithme génétique en réorganisant certaines arêtes de la tournée afin de réduire sa longueur totale.

Par ailleurs, la figure montre également que les deux méthodes convergent progressivement vers des solutions stables au fil des générations. Cependant, l'approche hybride atteint généralement des valeurs de coût plus faibles, ce qui confirme l'intérêt de combiner une méthode d'exploration globale (algorithme génétique) avec une méthode d'amélioration locale (2-opt).

7 Discussion

Les résultats expérimentaux montrent que l'hybridation entre l'algorithme génétique et l'heuristique d'amélioration locale 2-opt constitue une stratégie efficace pour résoudre le problème du voyageur de commerce.

L'algorithme génétique permet d'explorer efficacement l'espace des solutions grâce aux mécanismes de sélection, de croisement et de mutation. Cependant, les solutions générées peuvent encore être améliorées localement.

L'application de la recherche locale 2-opt permet alors d'améliorer ces solutions en réduisant la longueur des tournées obtenues. Cette combinaison permet ainsi de bénéficier à la fois de la capacité d'exploration globale des algorithmes évolutionnaires et de la précision des méthodes d'amélioration locale.

Toutefois, l'application de la recherche locale introduit un coût computationnel supplémentaire, notamment pour les instances de grande taille. Le compromis entre qualité des solutions et temps de calcul constitue donc un aspect important à prendre en compte lors de l'utilisation de ce type d'approche hybride.

8 Conclusion

Dans ce projet, plusieurs méthodes ont été étudiées pour résoudre le problème du voyageur de commerce (TSP). Trois approches principales ont été analysées : une heuristique constructive (Nearest Neighbor), un algorithme génétique, et une approche hybride combinant l'algorithme génétique avec une méthode d'amélioration locale 2-opt.

Les expérimentations réalisées sur plusieurs instances de la bibliothèque TSPLIB ont permis d'évaluer les performances de ces différentes méthodes. Les résultats montrent que l'algorithme génétique permet déjà d'obtenir des solutions satisfaisantes grâce à sa capacité à explorer un large espace de solutions.

Cependant, l'intégration de l'amélioration locale 2-opt dans l'approche hybride permet d'obtenir des solutions de meilleure qualité en affinant les tournées générées par l'algorithme génétique. Les courbes de convergence montrent également que cette approche permet d'atteindre plus rapidement des solutions stables et performantes.

La comparaison avec les solutions optimales de TSPLIB met en évidence que les solutions obtenues par l'approche hybride sont proches des valeurs optimales pour plusieurs instances, ce qui confirme l'efficacité de cette stratégie.

Ainsi, l'approche hybride constitue une méthode pertinente pour résoudre des instances de taille moyenne du problème du voyageur de commerce, en combinant exploration globale et amélioration locale.

9 Perspectives

Plusieurs pistes d'amélioration peuvent être envisagées afin d'améliorer encore les performances de l'approche proposée :

- l'utilisation d'heuristiques d'amélioration locale plus avancées telles que 3-opt ou Lin-Kernighan ;
- l'intégration d'autres métaheuristiques comme l'optimisation par colonies de fourmis (ACO) ;
- l'utilisation de méthodes de recherche tabou pour éviter certains minima locaux ;

— l'hybridation de plusieurs métaheuristiques afin d'améliorer encore la qualité des solutions obtenues.

10 Annexes

Les annexes contiennent les éléments nécessaires pour reproduire les expérimentations réalisées dans ce projet.

Le dossier `TSP.zip` est structuré de la manière suivante :

— `notebook.ipynb` : notebook contenant l'implémentation complète des algorithmes utilisés dans ce projet (algorithme génétique, amélioration locale 2-opt et approche hybride).

— `data/` : dossier contenant les instances du problème du voyageur de commerce utilisées pour les expérimentations. Ce dossier comprend les dix instances issues de la bibliothèque TSPLIB : `burma14`, `ulysses16`, `ulysses22`, `att48`, `eil51`, `berlin52`, `st70`, `eil76`, `kroA100` et `ch130`.

Ces fichiers permettent de reproduire les résultats expérimentaux présentés dans ce rapport.